Installations- und Administrationsdokumentation

Diese Dokumentation beschreibt die wichtigsten Systemvoraussetzungen und administrativen Schritte, die für den Betrieb der Anwendung **Study Planner** notwendig sind.

Anwendungstyp

Der Study Planner ist eine lokale Desktop-Anwendung für Studenten.

Eigenschaften:

- Java-Desktop-Anwendung
- Swing-Oberfläche
- lokale Datenspeicherung
- kein Server notwendig
- keine Cloud-Anbindung
- kein Mehrbenutzerbetrieb
- keine externe Datenbank notwendig

Systemvoraussetzungen

Betriebssystem

Die Anwendung ist für folgende Betriebssysteme geeignet:

- Windows
- Linux
- macOS

Die Anwendung wurde als Java-Desktop-Anwendung umgesetzt und ist daher grundsätzlich plattformunabhängig, solange eine passende Java-Version installiert ist.

Java

Es wird Java 17 oder neuer benötigt.

Empfohlen:

- Java Development Kit 17 oder neuer

Prüfen der Java-Version:

```
java -version
```

Prüfen des Java-Compilers:

```
javac -version
```

Wenn `java` oder `javac` nicht gefunden wird, muss ein JDK installiert und der `PATH` angepasst werden.

Maven

Das Projekt wird mit Maven gebaut und gestartet.

Empfohlen:

- Apache Maven 3.8 oder neuer

Prüfen der Maven-Version:

```
mvn -version
```

Wenn `mvn` nicht gefunden wird, muss Maven installiert und der Maven-`bin`-Ordner zum `PATH` hinzugefügt werden.

Speicherplatz

Die Anwendung selbst benötigt nur wenig Speicherplatz.

Empfohlen:

- mindestens 100 MB freier Speicherplatz für Projekt, Build-Dateien und lokale Daten

Beim ersten Maven-Build werden Abhängigkeiten heruntergeladen. Dadurch kann im lokalen Maven-Repository zusätzlicher Speicherplatz belegt werden.

Arbeitsspeicher

Empfohlen:

- mindestens 512 MB freier Arbeitsspeicher

Da es sich um eine einfache Desktop-Anwendung handelt, sind keine hohen Hardwareanforderungen notwendig.

Internetverbindung

Eine Internetverbindung ist nur beim ersten Maven-Build notwendig, wenn Maven die benötigten Abhängigkeiten herunterlädt.

Danach kann die Anwendung lokal gestartet werden, solange alle Abhängigkeiten bereits im lokalen Maven-Repository vorhanden sind.

Projektstruktur

Wichtige Ordner:

```
java_projekt/
      pom.xml
      src/
          main/
              java/
          test/
              java/
      target/
```

Beschreibung:

- `pom.xml`: Maven-Konfiguration
- `src/main/java`: Anwendungscode
- `src/test/java`: JUnit-Tests
- `target`: automatisch erzeugter Build-Ordner

Der Ordner `target` wird von Maven erzeugt und sollte nicht manuell versioniert werden.

Installation

Eine klassische Installation ist nicht notwendig.

Vorgehen:

1. Repository oder Projektordner auf den Zielrechner kopieren.
2. Java 17 oder neuer installieren.
3. Maven installieren.
4. Terminal im Ordner `java_projekt` öffnen.
5. Anwendung bauen oder starten.

Anwendung starten

Im Ordner `java_projekt`:

```
mvn exec:java
```

Dadurch wird die Anwendung direkt über Maven gestartet.

Anwendung bauen

Im Ordner `java_projekt`:

```
mvn package
```

Die erzeugte JAR-Datei liegt danach im Ordner:

```
target/
```

Die JAR-Datei kann gestartet werden mit:

```
java -jar target/study-planner-1.0.0.jar
```

Tests ausführen

Die automatisierten Tests werden mit JUnit 5 ausgeführt.

Im Ordner `java_projekt`:

```
mvn test
```

Ein erfolgreicher Testlauf endet mit:

```
BUILD SUCCESS
```

Javadoc erzeugen

Die technische HTML-Dokumentation kann mit folgendem Befehl erzeugt werden:

```
mvn javadoc:javadoc
```

Die erzeugte Dokumentation liegt danach unter:

```
target/site/apidocs/index.html
```

Datenspeicherung

Die Anwendung speichert Daten lokal im Benutzerordner.

Standardpfad:

```
<Benutzerordner>/.studyplanner/studyplanner.dat
```

Beispiele:

Windows:

```
C:\Users<Benutzername>.studyplanner\studyplanner.dat
```

Linux/macOS:

```
/home/<Benutzername>/.studyplanner/studyplanner.dat
```

Die Datei enthält die lokal gespeicherten Daten der Anwendung.

Backup

Für ein Backup sollte der Ordner `.studyplanner` im Benutzerordner gesichert werden.

Wichtige Datei:

```
studyplanner.dat
```

Zum Wiederherstellen kann diese Datei wieder an denselben Speicherort kopiert werden.

Löschen lokaler Daten

Wenn alle lokal gespeicherten Daten entfernt werden sollen, kann die Datei `studyplanner.dat` gelöscht werden.

Beim nächsten Start beginnt die Anwendung mit einem leeren Datenbestand.

Vor dem Löschen sollte bei Bedarf ein Backup erstellt werden.

Wichtige Maven-Abhängigkeiten

Die Anwendung nutzt:

- JUnit 5 für Tests
- Maven Compiler Plugin
- Maven Surefire Plugin
- Exec Maven Plugin
- Maven Javadoc Plugin
- Maven JAR Plugin

Diese Abhängigkeiten und Plugins werden in der `pom.xml` verwaltet.

Git-Hinweise

Folgende Ordner sollten nicht versioniert werden:

```
target/
  out/
```

Diese Ordner enthalten automatisch erzeugte Build-Dateien.

Typische Probleme

`java` wird nicht gefunden

Ursache:

- Java ist nicht installiert
- Java ist nicht im `PATH`

Lösung:

- JDK 17 oder neuer installieren
- `PATH` anpassen
- Terminal neu starten

`mvn` wird nicht gefunden

Ursache:

- Maven ist nicht installiert
- Maven ist nicht im `PATH`

Lösung:

- Maven installieren
- Maven-`bin`-Ordner zum `PATH` hinzufügen
- Terminal neu starten

Tests werden nicht gefunden

Ursache:

- Tests liegen nicht unter `src/test/java`
- Maven wird im falschen Ordner gestartet

Lösung:

- Terminal im Ordner `java_projekt` öffnen
- Prüfen, ob Testdateien unter `src/test/java` liegen
- Danach `mvn test` ausführen

Anwendung startet nicht

Mögliche Ursachen:

- falsche Java-Version
- Maven-Abhängigkeiten wurden noch nicht geladen
- Startbefehl wurde im falschen Ordner ausgeführt

Lösung:

```
java -version
  mvn -version
  mvn test
  mvn exec:java
```

Administrativer Ablauf vor Abgabe

Vor einer Abgabe oder Weitergabe der Anwendung sollte Folgendes geprüft werden:

1. Java-Version prüfen.
2. Maven-Version prüfen.
3. Tests ausführen:

```
mvn test
```

1. Anwendung starten:

```
mvn exec:java
```

1. Keine Build-Ordner wie `target/` oder `out/` mit abgeben.

Zusammenfassung

Für den Betrieb der Anwendung werden benötigt:

- Java 17 oder neuer
- Maven
- lokaler Speicherplatz
- beim ersten Build Internetzugang für Maven-Abhängigkeiten

Ein separater Server, eine Datenbank oder eine Cloud-Anbindung sind nicht notwendig.